

Balance a Binary Search Tree

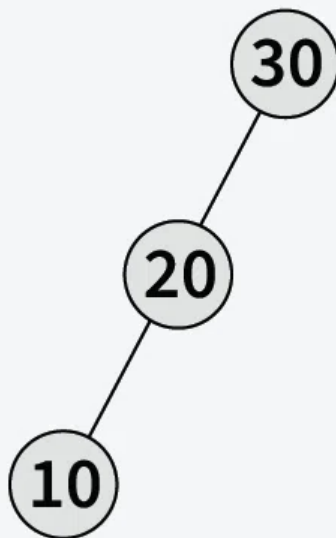
Last Updated : 05 Feb, 2025



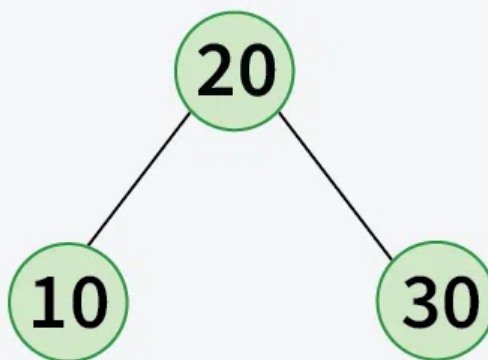
Given a **BST** (Binary Search Tree) that may be **unbalanced**, the task is to convert it into a **balanced BST** that has the **minimum** possible height.

Examples:

Input:

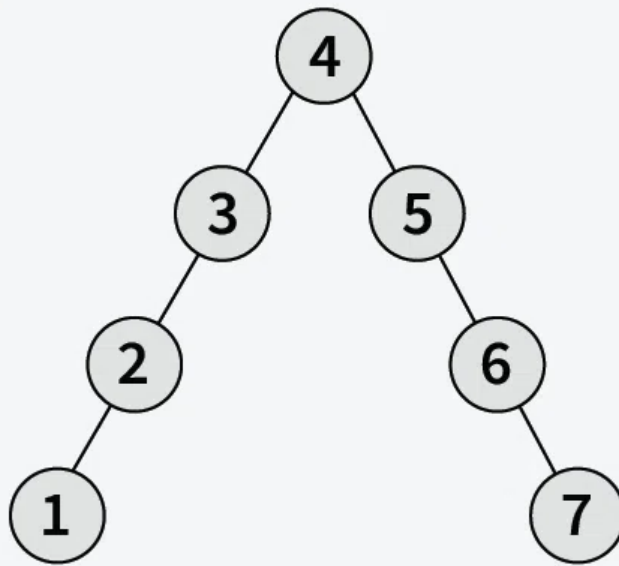


Output:

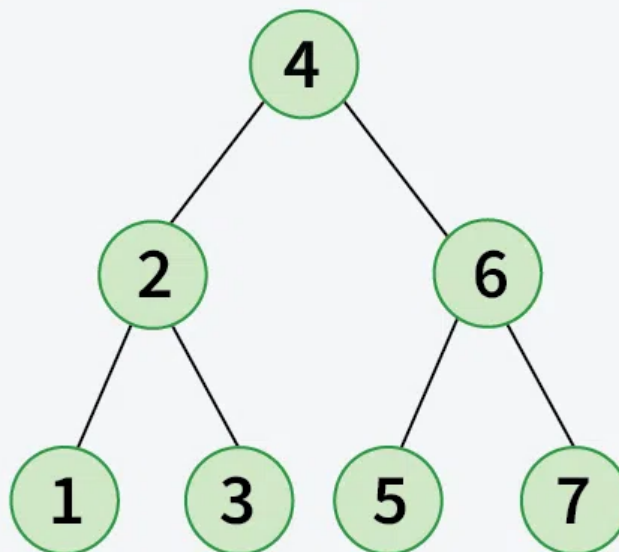


Explanation: The above unbalanced BST is converted to balanced with the minimum possible height.

Input:



Output:



Explanation: The above unbalanced BST is converted to balanced with the minimum possible height.

Approach:

The idea is to store the elements of the tree in an array using **inorder traversal**. Inorder traversal of a **BST** produces a **sorted array**. Once we have a sorted array, recursively construct a **balanced BST** by picking the **middle element** of the array as the root for each subtree.

Follow the steps below to solve the problem:

- Traverse given **BST** in **inorder** and store result in an **array**. Note that this array would be sorted as inorder traversal of BST always produces sorted sequence.

- Build a balanced BST from the above created sorted array using the **recursive** approach discussed in [Sorted Array to Balanced BST](#).

C++

Java

Python

C#

JavaScript

```
// Inorder traversal to store elements of the
// tree in sorted order
void storeInorder(Node* root, vector<int>& nodes) {
    if (root == nullptr)
        return;

    // Traverse the left subtree
    storeInorder(root->left, nodes);

    // Store the node data
    nodes.push_back(root->data);

    // Traverse the right subtree
    storeInorder(root->right, nodes);
}

// Function to build a balanced BST from a sorted array
Node* buildBalancedTree(vector<int>& nodes, int start, int end) {

    // Base case
    if (start > end)
        return nullptr;

    // Get the middle element and make it the root
    int mid = (start + end) / 2;
    Node* root = new Node(nodes[mid]);

    // Recursively build the left and right subtrees
    root->left = buildBalancedTree(nodes, start, mid - 1);
    root->right = buildBalancedTree(nodes, mid + 1, end);

    return root;
}
```

```
// Function to balance a BST
Node* balanceBST(Node* root) {
    vector<int> nodes;

    // Store the nodes in sorted order
    storeInorder(root, nodes);

    // Build the balanced tree from the sorted nodes
    return buildBalancedTree(nodes, 0, nodes.size() - 1);
}
```



Output

4 2 6 1 3 5 7

Time Complexity: $O(n)$, as we are just traversing the tree twice. Once in inorder traversal and then in construction of the balanced tree.

Auxiliary space: $O(n)$, as extra space is used to store the nodes of the inorder traversal in the vector. Also the extra space taken by recursion call stack is $O(h)$ where h is the height of the tree.

Balance a Binary Search Tree

[Visit Course ↗](#)

 Comment

More info ▼

Advertise with us

Next Article >

Self-Balancing Binary Search Trees

Similar Reads

Binary Search Tree

A Binary Search Tree (or BST) is a data structure used in computer science for organizing and storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right...

 3 min read

Introduction to Binary Search Tree

Binary Search Tree is a data structure used in computer science for organizing and storing data in a sorted manner. Binary search tree follows all properties of binary tree and for every nodes, its left subtree...

🕒 3 min read

Applications of BST

Binary Search Tree (BST) is a data structure that is commonly used to implement efficient searching, insertion, and deletion operations along with maintaining sorted sequence of data. Please remember th...

🕒 3 min read

Applications, Advantages and Disadvantages of Binary Search Tree

A Binary Search Tree (BST) is a data structure used to storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right child, with the left child containing...

🕒 2 min read

Insertion in Binary Search Tree (BST)

Given a BST, the task is to insert a new node in this BST.Example: How to Insert a value in a Binary Search Tree:A new key is always inserted at the leaf by maintaining the property of the binary search...

🕒 15 min read

Searching in Binary Search Tree (BST)

Given a BST, the task is to search a node in this BST. For searching a value in BST, consider it as a sorted array. Now we can easily perform search operation in BST using Binary Search Algorithm. Input: Root of...

🕒 7 min read

Deletion in Binary Search Tree (BST)

Given a BST, the task is to delete a node in this BST, which can be broken down into 3 scenarios:Case 1. Delete a Leaf Node in BSTDeletion in BSTCase 2. Delete a Node with Single Child in BSTDeleting a...

🕒 10 min read

Binary Search Tree (BST) Traversals – Inorder, Preorder, Post Order

Given a Binary Search Tree, The task is to print the elements in inorder, preorder, and postorder traversal of the Binary Search Tree.Â Input:Â A Binary Search TreeOutput:Â Inorder Traversal: 10 20 30 100 150...

🕒 10 min read

Balance a Binary Search Tree

Given a BST (Binary Search Tree) that may be unbalanced, the task is to convert it into a balanced BST that has the minimum possible height.Examples: Input: Output: Explanation: The above unbalanced BST...

Self-Balancing Binary Search Trees

Self-Balancing Binary Search Trees are height-balanced binary search trees that automatically keep the height as small as possible when insertion and deletion operations are performed on the tree. The heigh...